

10b_code_exact_change_spec

10b — LLMsVerifier Code-Exact Change Spec + Build/Test Baseline (T1/main)

Date: 2026-07-06 **Scope:** Ground the P2-T2/T3 LLMsVerifier extension in the ACTUAL current code of submodules/llms_verifier (NOT the vendored claude_toolkit copy). Every struct below is pasted verbatim from source at the cited file:line; every build/test claim carries captured terminal output (§11.4.5 / §11.4.6 — no guessing). Complements 10_llmsverifier_helixagent.md (which summarized these structs; this doc pins exact line numbers + adds the physical build/test baseline + precise change sites).

Evidence dir: docs/research/07.2026/00_master/evidence/llmsverifier_baseline/ (go_build.txt, go_test.txt).

0. Repo / module / git state (FACT)

Fact	Value	Source
Submodule path	submodules/llms_verifier	.gitmodules grouped layout
Git HEAD	62d68c77 merge: reconcile dangling-reference fix with upstream semantic-code-visibility work	git log --oneline -1
Behind/ahead vs origin/main	0 0 (local tracking ref; NO network fetch performed)	git rev-list --left-right --count HEAD...origin/main
Commit 0e7d6949 (claude_toolkit report's "1 behind")	NOT present in this submodule's object store (git cat-file -t 0e7d6949 → missing)	captured
Recent lineage	62d68c77 ← 3e7018f7 fix: remove dangling reference...	git log --oneline -3

Fact	Value	Source
	← 7a3ae9a6 merge: reconcile DB-persistence fix (044768e0)...	
Root module	module llmsverifier (go 1.25.3) — thin wrapper	go.mod:1
Inner app module	module digital.vasic.llmsverifier (go 1.25.3)	llm-verifier/ go.mod:1
Go toolchain used	go1.26.4-X:nodwarf5 linux/ amd64	go_build.txt

UNCONFIRMED (git remote-state): the 0 0 behind/ahead is against the *local* origin/main tracking ref — NO git fetch was run this session (no network), so whether the true remote advanced is unknown per §11.4.6/§11.4.37. What IS confirmed: this submodule’s HEAD 62d68c77 and the claude_toolkit-report anchor 0e7d6949 are on **different histories** — 0e7d6949 is not reachable in this checkout. The two copies are NOT the same lineage; “1 commit behind 0e7d6949” does not describe THIS submodule. A fetch is required before any pull/rebase decision.

1. Physical build/test baseline (captured this session)

1.1 go build ./... — COMPILES (exit 0)

From go_build.txt (run from the submodule ROOT, module llmsverifier):

```
=== go build ./... (started 2026-07-05T22:07:49Z) ===
go: downloading github.com/segmentio/kafka-go v0.4.50
go: downloading github.com/rabbitmq/amqp091-go v1.10.0
go: downloading golang.org/x/sys v0.46.0
... (transitive deps)
=== go build EXIT=0 (finished 2026-07-05T22:08:05Z) ===
```

go build ./... from the submodule ROOT (module llmsverifier) returns **exit 0**. NOTE: this builds ONLY the thin root module’s packages; it does NOT descend into the nested inner module llm-verifier/ (digital.vasic.llmsverifier). The inner build that DOES cover the change-site packages is cd llm-verifier && go build ./... = **exit 0** (§5.3, go_test_inner.txt). Both green today.

1.2 go test -short ./... — baseline

See §5 (filled from captured go_test.txt after the run completes). Command: GOMAXPROCS=2 GOFLAGS=-mod=mod nice -n 19 go test -short ./... (resource-capped per inner CLAUDE.md rule 9). Evidence: go_test.txt.

2. Current struct shapes — pasted verbatim (FACT)

2.1 Message — CONFIRMED NO ToolCalls field (the stream-10 finding)

llm-verifier/llmverifier/llm_client.go:91-95:

```
// Message represents a message in the chat completion
type Message struct {
    Role    string `json:"role"`
    Content string `json:"content"`
}
```

Two fields only — Role, Content. **No ToolCalls**. The sibling request/response types in the same file confirm the gap end-to-end:

- ChatCompletionRequest (llm_client.go:80-89) HAS Tools []Tool + ToolChoice interface{} — so the client CAN *send* tools...
- ...but ChatCompletionChoice (llm_client.go:98-102) embeds Message Message — so a tool-call in the *response* has nowhere to land. The model's tool_calls array is silently dropped at unmarshal.

2.2 VerificationService.Verify — UNWIRED, returns ErrVerificationNotWired (FACT)

llm-verifier/verification/verification.go:29-63 (body abridged to the operative return; full comment block is lines 42-61):

```
func (v *Verifier) Verify(ctx context.Context, req *Request)
    (*database.VerificationResult, error) {
    if req == nil { return nil, fmt.Errorf("verification request
        cannot be nil") }
    if req.ModelID == "" { return nil, fmt.Errorf("model ID is
        required") }
    if req.Prompt == "" { return nil, fmt.Errorf("prompt is
        required") }
    // ... [42-61] previously returned a hardcoded all-caps-
        true / all-scores-8.5
```

```

//      VerificationResult regardless of model – a
//      CONST-036/037 PASS-bluff, now removed.
return nil, ErrVerificationNotWired
}

```

llm-verifier/verification/verification.go:69:

```

var ErrVerificationNotWired =
    fmt.Errorf("verification.VerifyModel: real verifier
    dispatch is not wired (model resolution + per-feature
    test + score composition required); the previous
    hardcoded-all-capabilities-true return was a §11.4 /
    CONST-036/037 PASS-bluff and is now removed – wire
    llmverifier.Verifier into VerificationService to
    restore")

```

Verifier holds only db *database.Database (verification.go:17-19); the real per-feature test engine (llmverifier.Verifier, llmverifier/verifier.go) is NOT referenced here. Restoring verification = wiring llmverifier.Verifier into this VerificationService. Guarded by verification/verification_test.go:64-101 (asserts errors.Is(err, ErrVerificationNotWired)).

2.3 Tool-calling counter — honestly returns (false, 0) (FACT)

llm-verifier/llmverifier/verifier.go:1042 (signature) and :1093 / :1111 (returns):

```

// testParallelToolUse checks for parallel tool use capability
func (v *Verifier) testParallelToolUse(client *LLMClient,
    modelName string, ctx context.Context) (bool, int) {
    req := ChatCompletionRequest{ Model: modelName, Messages:
        []Message{{Role:"user", Content:"..."}}, Tools: []Tool{ /
        * get_weather, get_stock_price */ }, ToolChoice: "auto" }
    resp, err := client.ChatCompletion(ctx, req)
    if err != nil { return false, 0 } //
    :1093
    // §11.4 / CONST-035 – Counting REAL tool calls requires
    // parsing the
    // OpenAI-shape `tool_calls` array on every Choice.Message,
    // but this
    // codebase's Message struct (llm_client.go:92) does NOT yet
    // carry a
    // ToolCalls field ... Until the schema lands, return the
    // honest
    // sentinel (false, 0). [Previously hardcoded toolCallCount
    // = 2.]

```

```

    _ = resp
    return false, 0 //
        :1111
}

```

The in-code comment (verifier.go:1096-1109) itself names llm_client.go:92 as the blocker and calls the extension a “CONST-039 prerequisite.” This is the load-bearing dependency: **parallel/tool-use detection cannot function until Message.ToolCalls exists.**

2.4 Capability model — capabilities/types.go (FACT)

ProviderCapabilities (capabilities/types.go:82-114) — the rich, verification-fillable set; the two CONST-040-relevant members:

```

type ProviderCapabilities struct {
    Provider    string    `json:"provider"`
    Model       string    `json:"model,omitempty"`
    Verified    bool      `json:"verified"`
    VerifiedAt  time.Time `json:"verified_at,omitempty"`
    Streaming   StreamingCapability `json:"streaming"`
    Network     NetworkCapability   `json:"network"`
    Compression CompressionCapability `json:"compression"`
    Caching     CachingCapability   `json:"caching"`
    Protocols   []ProtocolType      `json:"protocols" //
        MCP/ACP/LSP/gRPC/OpenAI/... as TAGS
    Auth        AuthCapability       `json:"auth"`
    Model_      ModelCapability      `json:"model_capabilities"`
    Extended    ExtendedCapabilities  `json:"extended"`
    Custom      map[string]interface{} `json:"custom,omitempty"`
}

```

ModelCapability (capabilities/types.go:168-184) — verbatim:

```

type ModelCapability struct {
    Vision          bool `json:"vision"`
    ImageInput      bool `json:"image_input"`
    ImageOutput     bool `json:"image_output"`
    Audio           bool `json:"audio"`
    Video           bool `json:"video"`
    PDF             bool `json:"pdf"`
    OCR             bool `json:"ocr"`
    FunctionCalling bool `json:"function_calling"`
    ToolUse         bool `json:"tool_use"`
    Embeddings      bool `json:"embeddings"`
    CodeExecution   bool `json:"code_execution"`
}

```

```

WebBrowsing      bool    `json:"web_browsing"`
Reasoning        bool    `json:"reasoning"`
MaxContextTokens int     `json:"max_context_tokens"`
MaxOutputTokens  int     `json:"max_output_tokens"`
}

```

Finding: MCP/ACP/LSP live only as Protocols []ProtocolType tags (not per-model booleans); ModelCapability has **NO RAG / Skills / Plugins member**, and no CapabilityEvidence. CONST-040 requires MCP, LSP, ACP, Embedding, RAG, Skills, Plugins — today only Embedding (Embeddings) + MCP/LSP/ACP (as tags) exist here.

2.5 registry.go — the static protocol-flag block (CONST-040 anti-bluff hazard, FACT)

llm-verifier/capabilities/registry.go is a package-level var
 providerCapabilities = map[string]*ProviderCapabilities{...}
 (registry.go:8-427) — **8 hardcoded providers** (openai, anthropic, deepseek, gemini, qwen, groq, mistral, zen, ollama) plus cliAgentCapabilities
 (registry.go:430-1065). Every Protocols: and Model_: value is a STATIC LITERAL, e.g.:

- registry.go:38 — openai: Protocols:
 []ProtocolType{ProtocolOpenAI}
- registry.go:90 — anthropic: Protocols:
 []ProtocolType{ProtocolAnthropic, ProtocolMCP}
- registry.go:44-54 — openai Model_:
 ModelCapability{Vision:true, FunctionCalling:true,
 ToolUse:true, Embeddings:true, Reasoning:true, ...}
- registry.go:102 — anthropic FunctionCalling: false (hand-authored, never probed)

Accessors GetProviderBaseCapabilities (registry.go:1067), GetAllProviders (registry.go:1084), GetProvidersWithCapability (registry.go:1101) all read the static map directly — **no probe path, no verifier override, no freshness check**. This is the CONST-036/037/040 hazard: consumers calling these get hand-authored flags presented as authoritative, never verified within 24h (CONST-037) nor sourced from a VerificationResult (CONST-040).

2.6 Per-model DB record — database/database.go (already CONST-040-shaped, FACT)

database.VerificationResult (database/database.go:875-940) already carries the per-model Supports* booleans + freshness columns:

```

type VerificationResult struct {
    ID int64; ModelID int64; VerificationType string
    StartedAt time.Time; CompletedAt *time.Time; Status string;
    ErrorMessage *string // freshness
    ModelExists, Responsive, Overloaded *bool; LatencyMs *int
    SupportsToolUse, SupportsFunctionCalling bool
    SupportsCodeGeneration, SupportsCodeCompletion,
    SupportsCodeReview, SupportsCodeExplanation bool
    SupportsEmbeddings, SupportsReranking bool
    SupportsImageGeneration, SupportsAudioGeneration,
    SupportsVideoGeneration bool
    SupportsMCPs, SupportsLSPs, SupportsACPs bool //
    :898-900 CONST-040 booleans EXIST
    SupportsMultimodal, SupportsStreaming, SupportsJSONMode,
    SupportsStructuredOutput bool
    SupportsReasoning, SupportsParallelToolUse bool;
    MaxParallelCalls int
    SupportsBatchProcessing, SupportsBrotli bool
    CodeLanguageSupport []string; /* ...code/score/latency/raw
    fields... */
    RawRequest, RawResponse *string; CreatedAt time.Time
}

```

- SQL DDL supports_mcps BOOLEAN DEFAULT 0 at database/database.go:538 (+ sibling supports_lsps/supports_acps cols).
- Persistence writes them: database/database.go:1224 (INSERT column list) + :1261-1263 (bind SupportsMCPs/LSPs/ACPs); mirror path in database/crud.go:620,657-659,722,764-766,836,925-927.

Finding: the DB layer already STORES + PERSISTS SupportsMCPs/LSPs/ACPs/Embeddings/... and freshness (StartedAt/CompletedAt/Status). It has **NO supports_rag / supports_skills / supports_plugins column** and **no RawRequest/RawResponse per-capability evidence linkage** beyond the single overall pair. Gap = population (the Verify unwiring §2.2) + 3 missing columns, not architecture.

2.7 In-memory verification result — llmverifier/models.go:10 (FACT, from 10_report)

llmverifier.VerificationResult (llmverifier/models.go:10) + Capabilities (llmverifier/models.go:83) carry Completion/Chat/Embedding/ToolUse/Multimodal/ FunctionCalling/... — the runtime shape the probe engine fills, distinct from the DB record. (Full paste in 10_llmsverifier_helixagent.md §1.3.)

3. PRECISE change spec (every site cited to real file:line)

C1 — Add ToolCalls to Message (unblocks §2.1/§2.3)

Site: llm-verifier/llmverifier/llm_client.go:92-95. Add a ToolCalls field + the supporting ToolCall types (OpenAI shape):

```
type Message struct {
    Role      string    `json:"role"`
    Content   string    `json:"content"`
    ToolCalls []ToolCall `json:"tool_calls,omitempty" // NEW
}

// NEW types (same file):
type ToolCall struct {
    ID      string    `json:"id"`
    Type    string    `json:"type"`
    "function"
    Function ToolCallFunction `json:"function"`
}

type ToolCallFunction struct {
    Name      string    `json:"name"`
    Arguments string    `json:"arguments" // raw
    JSON string per OpenAI
}
```

Downstream unblock: testParallelToolUse(verifier.go:1042, currently returns (false,0) at :1111) becomes implementable — replace _ = resp; return false, 0 with a real count of resp.Choices[i].Message.ToolCalls (>1 ⇒ parallel). testToolUse(verifier.go:698) likewise gains a real signal.

Blast radius: Message is also declared in 5 OTHER packages (providers/openai.go:156, testsuite/builder.go:140, enhanced/adapters/providers.go:16, enhanced/context/short_term.go:10, enhanced/validation/gates.go:20, internal/messaging/broker.go:30) — these are DISTINCT types, NOT shared; only the llmverifier one is on the tool-use path. Verify no cross-import before/after (per §11.4.92 Pass 2).

C2 — Add RAG/Skills/Plugins + evidence to the capability model

Site A — capabilities/types.go:168-184 (ModelCapability): append


```

SupportsRAG      bool    `json:"supports_rag"`           // NEW
(CONST-040)
SupportsSkills   bool    `json:"supports_skills"`       // NEW
(CONST-040)
SupportsPlugins  bool    `json:"supports_plugins"`      // NEW
(CONST-040)

```

Site B — capabilities/types.go:82-114 (ProviderCapabilities): add a per-capability evidence map so every flag traces to a captured probe artefact (§11.4.5/§11.4.69):

```

CapabilityEvidence map[string]CapabilityEvidenceEntry
`json:"capability_evidence,omitempty"` // NEW

```

with a NEW type (same file): CapabilityEvidenceEntry{ ProbedAt time.Time; Method string; RawRequestPath, RawResponsePath string; Verdict bool }.

Site C — database/database.go:875-940 (database.VerificationResult): add SupportsRAG, SupportsSkills, SupportsPlugins bool columns + DDL at database/database.go:538-region (supports_rag/skills/plugins BOOLEAN DEFAULT 0) + INSERT/scan in database/database.go:1224/1261-1263 AND database/crud.go:620/657-659/722/764-766/836/925-927 (both persistence paths — miss one and reads/writes desync).

C3 — Demote registry.go static block: seed-only, probe-overridden, fail-closed

Site: llm-verifier/capabilities/registry.go:8-427

(providerCapabilities map) + accessors

GetProviderBaseCapabilities:1067, GetProvidersWithCapability:1101.

Change: the static map becomes a **SEED / bootstrap default ONLY**, never the authoritative answer to a capability query: 1. Rename intent: mark the map providerCapabilitySeeds (doc-comment: “hand-authored bootstrap defaults; NOT verified — MUST be overridden by a live probe / DB VerificationResult before being shown to any user, per CONST-036/037/040”). 2. Route every accessor through a resolver that prefers a fresh database.VerificationResult (per model, within the CONST-037 24h / CONST-038 60s window using VerificationResult.StartedAt/CompletedAt); the seed is used ONLY when no probe exists. 3. **Fail-closed:** when neither a fresh probe nor a seed is available, return unknown/unverified (NOT a silent false, NOT a stale seed) — a capability the verifier has not confirmed is reported as unverified, never as a hand-authored literal. 4. Static FunctionCalling:false-style literals (e.g. anthropic

registry.go:102) are demoted to seeds; the live probe (C4) is the source of truth.

C4 — Per-capability probe list (each emits a captured wire artefact)

The probe engine (llmverifier/verifier.go) already has testToolUse:698, testParallelToolUse:1042, testBatchProcessing:1115, etc. Add/complete one probe per CONST-040 capability, each writing its raw request+response to a captured artefact path (feeds C2 Site B CapabilityEvidence + DB RawRequest/RawResponse):

Capability	Probe (real wire call)	Positive-evidence shape
ToolUse / FunctionCalling	send tools+tool_choice, assert resp.Choices[].Message.ToolCalls non-empty (needs C1)	captured request+response JSON showing a tool_calls entry
ParallelToolUse	2-tool prompt, assert len(ToolCalls) > 1 (needs C1)	response with ≥2 tool_calls
Embeddings	POST /v1/embeddings, assert vector len>0	response with non-empty data[].embedding
MCP	provider MCP endpoint handshake / capability advertise	captured handshake showing MCP support
LSP	LSP capability probe	captured advertise
ACP	ACP capability probe	captured advertise
RAG	retrieval-augmented call (file/context injection round-trip)	response citing injected context
Skills	skills/agent-tool invocation round-trip	captured skill invocation
Plugins	plugin invocation round-trip	captured plugin call

Each probe: real HTTP (no simulation, per §3 of root CLAUDE.md), resource-capped, writes RawRequest/RawResponse to the DB record + a CapabilityEvidenceEntry, and stamps VerifiedAt/StartedAt/CompletedAt for the CONST-037/038 freshness window. Missing key = clean SKIP (per inner CLAUDE.md Acceptance demo), never a faked PASS.

C5 — Wire VerificationService.Verify (removes ErrVerificationNotWired)

Site: llm-verifier/verification/verification.go:29-63. Give Verifier a handle to llmverifier.Verifier (add field at verification.go:17-19), then in Verify: (1) resolve req.ModelID → int64 PK via the models table; (2) dispatch the C4 probes via llmverifier.Verifier; (3) compose a database.VerificationResult from real outcomes (incl. the new RAG/Skills/Plugins + evidence). Remove the return nil, ErrVerificationNotWired (:62) once real dispatch lands; UPDATE the guard test verification/verification_test.go:64-101 in the SAME commit (it currently asserts the sentinel) — this is a §11.4.120 fix-breaks-its-own-gate reconciliation, NOT a delete.

4. Ordering / dependency notes (for the executor)

1. **C1 first** — Message.ToolCalls is the keystone; C3/C4 tool-use probes and §2.3 depend on it. Landing C1 alone flips testParallelToolUse from a hard (false, 0) stub into an implementable probe.
2. **C2 before C4** — the evidence/columns must exist before probes can persist to them.
3. **C4 before C3's fail-closed resolver** and **before C5** — probes are the source of truth the resolver prefers and the service dispatches.
4. **C5 + test reconciliation last** — flipping Verify on requires the test update in the same commit (§11.4.120).

Every change site is inside the inner module digital.vasic.llmsverifier (llm-verifier/...), which builds clean today (§1.1) — a green build baseline to regress against.

5. Captured test baseline

5.1 ROOT module llmsverifier — go test -short ./... = GREEN (exit 0)

go_test.txt (run from submodule root; NOTE this covers ONLY the thin root module's package tree llmsverifier/..., NOT the inner app):

```
ok      llmsverifier      0.005s [no tests to run]
?       llmsverifier/challenges/runner [no test files]
ok      llmsverifier/internal/benchmark 12.369s
ok      llmsverifier/internal/llmops    2.574s
ok      llmsverifier/internal/messaging 0.131s
```

```

ok      llmsverifier/internal/messaging/factory 0.104s
ok      llmsverifier/internal/messaging/inmemory 1.380s
ok      llmsverifier/internal/messaging/kafka 0.009s
ok      llmsverifier/internal/messaging/rabbitmq 0.005s
ok      llmsverifier/internal/rag 0.006s
ok      llmsverifier/internal/selfimprove 0.006s
ok      llmsverifier/tests/e2e 0.006s
ok      llmsverifier/tests/integration 0.005s
ok      llmsverifier/tests/performance 0.004s
ok      llmsverifier/tests/security 4.027s
ok      llmsverifier/tests/unit 0.012s
=== go test EXIT=0 ===

```

15 ok, 0 FAIL, 1 no-test, exit 0. (Only cgo -Wdiscarded-qualifiers warnings from vendored mattn/go-sqlite3 — harmless.)

5.2 CRITICAL SCOPE FINDING — the root run does NOT test the change sites

go build ./.../go test ./... from the submodule ROOT operate on module llmsverifier (go.mod:1). Every change site in §3 (C1–C5) lives in llm-verifier/... which is a **SEPARATE module** digital.vasic.llmsverifier (llm-verifier/go.mod:1) with its own .git-independent module graph. go test ./... does NOT descend into a nested module — so the green §5.1 baseline says NOTHING about llmverifier/, capabilities/, verification/, database/. The inner baseline (§5.3, evidence go_test_inner.txt) is the one to regress the C1–C5 work against.

5.3 INNER module digital.vasic.llmsverifier — baseline

Command: cd llm-verifier && GOMAXPROCS=2 GOFLAGS=-mod=mod nice -n 19 go build ./... then go test -short ./.... Evidence: go_test_inner.txt.

- **go build ./... = exit 0** (inner app compiles clean — green baseline to regress C1–C5 against).
- **go test -short ./... = exit 1 — 58 ok, 3 FAIL.** Breakdown:

ALL five change-site packages are **GREEN**:

```

ok      digital.vasic.llmsverifier/capabilities 0.005s (registry.go /
ok      digital.vasic.llmsverifier/database      0.779s (VerificationR
ok      digital.vasic.llmsverifier/llmverifier   4.089s (llm_client.go
ok      digital.vasic.llmsverifier/providers     8.991s
ok      digital.vasic.llmsverifier/verification 0.143s (Verify / ErrV

```

The 3 FAILs are **environmental, pre-existing, and NOT on any change site** — all in `digital.vasic.llmsverifier/tests` (`automation_test.go`), CLI tests that require a running server at `https://localhost:8080`:

```
--- FAIL: TestCommandFlagValidation (automation_test.go:249)
--- FAIL: TestOutputFormats (automation_test.go:285)
    Output: Failed to fetch models: request failed: Get "https://localhost:8080": failed to verify certificate: x509: certificate is not valid
    but wanted to match localhost
FAIL digital.vasic.llmsverifier/tests 4.610s
```

These are the anti-bluff “real infrastructure or skip” tests firing WITHOUT the server up (should arguably SKIP, not FAIL, per inner CLAUDE.md rule 11 — a separate pre-existing hygiene issue, NOT introduced by this work).

Baseline verdict for the C1–C5 executor: inner module compiles; the packages you will edit (capabilities, database, llmverifier, verification) are all green today; the only red is an env-dependent CLI-server test unrelated to the change surface. Regress against `cd llm-verifier && go test -short ./capabilities/... ./database/... ./llmverifier/... ./verification/... (all currently PASS).`

Sources verified

- Code: all file:line above read directly from submodules/`llms_verifier/llm-verifier/...` this session (2026-07-06); structs pasted verbatim.
- Build/test: `docs/research/07.2026/00_master/evidence/llmsverifier_baseline/{go_build.txt,go_test.txt}` captured this session.
- Git state: `git log/rev-list/cat-file` captured this session.